Swarm Search

A Multi-Agent Coverage RL Environment

Project explainer: concept, environment design,
and the OpenStreetMap obstacle integration

Generated as a project reference document

The Concept

This project simulates a swarm of drones searching a grid-based area to maximize coverage as a team. It's built as a reinforcement learning (RL) environment: a policy (the thing being trained) controls each drone, and the environment rewards it for exploring efficiently.

Why grid coverage?

Grid coverage is a simplified stand-in for real search-and-rescue or surveying missions: send multiple drones over an area, avoid obstacles (buildings, water, terrain), and make sure the whole area gets seen at least once, as fast as possible, without needless overlap.

Why multi-agent?

A single drone covering a big area is slow. Multiple drones can split up and cover more ground in parallel, but only if they learn to spread out instead of following each other around. That coordination problem is exactly what the reward function in this environment is designed to teach.

The tech stack

- PettingZoo: the standard Python API for multi-agent RL environments. It defines a common contract (reset/step/observation_space/action_space) so any multi-agent RL library can plug in.
- Gymnasium: supplies the observation/action space types (Box, Discrete) used to describe what each drone can see and do.
- PyTorch / TorchRL (installed in the venv): where the actual neural network policy would be trained once you build the training loop.
- matplotlib: used for the live visualizer, and now for this PDF.

How the Grid Works

Everything happens on a grid_size x grid_size grid of cells. Each cell is in exactly one of these states:

- Obstacle -- can't be entered (a building, water, or out of bounds).
- Covered -- a drone has visited this cell at some point this episode.
- Uncovered -- free space nobody has reached yet. This is what the swarm is trying to eliminate.

Episode lifecycle

- reset(): obstacles are placed (randomly, or now from real OSM data -- see page 5), each drone gets a random free starting cell, and the 'covered' set starts out containing just those starting cells.
- step(actions): every drone simultaneously picks one of 5 actions -- up, down, left, right, or stay. Moves into an obstacle or off the grid are rejected and the drone just stays put instead.
- Each newly-visited cell gets added to the shared 'covered' set.
- The episode truncates after max_steps regardless of outcome.

What each drone actually 'sees' (the observation)

A drone doesn't see the whole grid -- that wouldn't scale, and it's not realistic (a real drone only has local sensors). Instead each drone gets:

- A local patch: a (2*obs_window+1) x (2*obs_window+1) square centered on itself, where each cell reads -1 (obstacle/out-of-bounds), 1 (covered), or 0 (uncovered free space). With the default obs_window=3, that's a 7x7 = 49-value patch.
- Its own normalized position: (row / grid_size, col / grid_size), so it knows roughly where in the world it is.

These are concatenated into one flat vector of length 49 + 2 = 51 -- this is exactly what gets fed into the neural network policy during training.

The reward: how the swarm learns to spread out

```
R_t = (number of newly covered cells this step) - step_penalty
```

This reward is shared identically across all drones (a 'team' reward, not individual). If any drone reaches a new cell, every drone gets credit. The small step_penalty (0.02 by default) discourages idling or wasting steps -- without it, 'stay' would look free.

Because reward comes from NEW coverage only, two drones sitting on top of each other stop generating reward -- so a trained policy is pushed toward spreading out and dividing the territory, without that being hand-coded anywhere.

How It Trains

The environment itself (swarm.py) doesn't train anything -- it's just the 'world'. Training means wiring a policy network up to it via a standard RL loop:

```
for episode in range(N):
    obs, infos = env.reset()
    while env.agents:
        actions = {a: policy(obs[a]) for a in env.agents}
        obs, rewards, terms, truncs, infos = env.step(actions)
        buffer.store(obs, actions, rewards)
    policy.update(buffer)   # e.g. PPO
```

Because SwarmCoverageEnv implements PettingZoo's ParallelEnv interface, it plugs directly into TorchRL's PettingZoo wrapper (already installed alongside torch in this venv) instead of needing a hand-rolled training loop. TorchRL handles the rollout collection, replay buffer, and the actual policy-gradient update (commonly PPO for this kind of problem).

Why the design choices matter for training

- Shared reward -> encourages cooperative spreading-out behavior instead of every drone learning an identical, redundant policy.
- Local-only observation -> forces the policy to learn something generalizable (react to nearby obstacles/coverage) rather than memorizing absolute positions on one fixed map.
- Randomized obstacles each reset -> without this, a policy could memorize one map instead of learning a real search strategy. This is why real OSM obstacles (page 5) are fixed per place rather than randomized -- they represent training/testing on one specific real environment, similar to how you'd hold out a validation map.
- Discrete action space (5 actions) -> keeps the policy network's output layer simple, appropriate for a first working RL loop before trying anything more complex like continuous flight control.

A natural next step: curriculum

Train on randomized grids first (fast, lots of variety, cheap to generate), then fine-tune or evaluate on real OSM-derived maps of specific places. That mirrors sim-to-real training patterns used in actual robotics/drone RL work.

The Changes: Real-World Obstacles

The most recent change replaces (optionally) the random obstacle layout with real building/water/forest footprints pulled from OpenStreetMap, so the swarm can search an actual place instead of an abstract random grid.

New file: swarm-env/osm_obstacles.py

- obstacles_from_bbox(bbox, grid_size, tags=None): given a lon/lat bounding box (west, south, east, north), downloads matching OSM features via the osmnx library and rasterizes them onto a grid_size x grid_size grid -- a cell becomes an obstacle if it overlaps any real-world feature geometry.
- Default tags cover buildings, water, and forest/wood -- i.e. things a drone genuinely could not fly through or shouldn't.
- Downloads are cached to .osm_cache/ (as GeoJSON, keyed by a hash of the bbox + tags) since OpenStreetMap's Overpass API is rate-limited and slow -- repeat runs against the same area reuse the cache.
- Also returns cell_bounds(row, col), a helper that converts a grid cell back into real lon/lat bounds -- useful later for drawing drones on an actual map view.

Changed: swarm-env/swarm.py

- SwarmCoverageEnv now accepts optional bbox and osm_tags parameters.
- When bbox is set, reset() uses that fixed real-world obstacle layout instead of scattering n_obstacles randomly. It stays the same every episode, since it represents one real, specific place.
- When bbox is omitted (the default), behavior is completely unchanged from before -- random obstacles every reset.

Changed: visualize_swarm.py

- demo_rollout() now accepts an optional bbox and forwards it to the environment, so the existing matplotlib visualizer can render a real-place run with no other changes.

New dependencies installed (swarm-env venv only)

```
osmnx, geopandas, shapely, pyproj, pyogrio, pandas, requests
```

These were verified to install and run correctly on the venv's Python 3.14, and a live Overpass API query was tested successfully.

Verified

- Random-obstacle path still produces exactly n_obstacles cells (no regression).
- A real bbox near UC Berkeley correctly returned real building obstacles rasterized onto a 20x20 grid.
- Obstacles from a real bbox stay identical across multiple resets.
- step() runs cleanly on an OSM-backed environment.

Open item to revisit

The first tested bbox was dense downtown blocks, so ~74% of the grid came back as obstacles -- barely any open space to search. A looser bbox (a park, a campus quad) or a coarser grid_size will give the swarm more room to actually move.